

Web Application Frameworks & Django

Internet Technology
ITECH

Week 2 Learning Objectives

- By the end of Week 2 you should:
 - Understand the **complexity of web development** and why we need **frameworks**
 - Map the elements of your **architecture design** to **Django modules**
 - With a little help of the Tango with Django book and your own study



Web Dev Complexity

Web Development Complexity

Collision of Languages

- Markup Languages
- Programming Languages
- Database Query Languages

Shifting Standards

- Document Object Model
- XML/JSON

Web Browser Compatibility

- Browser wars encourage new 'unique' features

HTTP is a Stateless Protocol

- But most applications require the persistence of state

Signs of Hope

Web development has become a serious business

In line with this, the methods of development are maturing and increasingly adopting good practices of 'classical' Software Engineering:

- Application Programmer Interfaces (APIs)
- Libraries
- Frameworks
- Tools
- Standards

Tools



Web Development Tools

Web development tool support is not yet as advanced as with classic software development

- Most languages have several complex IDEs (Integrated Development Environments) to choose from
 - Eclipse, .Net Framework, Django, etc

The nature of web development is disjoint

- a developer must become familiar with a set of distinct and (typically) non-integrated tools

What you code in...

An important tool is the IDE

Its choice and your expertise in its usage can seriously affect your productivity

- Syntax Awareness
- Auto-completion
- Snippets
- Scripts & Macros
- Integration with other development tools (revision control)

Some IDEs have plugins for scripting languages (e.g. Eclipse has a PHP plug-in)

Frameworks

- Frameworks provide design and partial implementations for a particular domain of applications
 - Drawing Applications (provide drawing primitives)
 - Distributed Middleware
 - Web Applications
- Frameworks allow developers to create applications more efficiently by providing **default functionality**, whilst allowing them to **extend and override** to suit their specific purposes

Classic Framework Definitions

- There are several interpretations of a framework:
 - A framework is a set of classes that embodies an **abstract design** for solutions to a **family of problems**
 - A framework is the **reusable design** of a system or a part of a system expressed as a set of **abstract classes** and the way the instances of those classes collaborate
 - A framework is a set of **prefabricated** software building **blocks** that programmers can **use, extend, or customize** for specific computing solutions;
 - Frameworks are large **abstract applications** in a **particular domain** that can be **tailored** for individual applications
 - A framework is a **reusable software architecture** comprising both **design and code**

Framework Characteristics

- **Inversion of Control**
 - Framework is responsible for the application control flow
- **Default Behaviour**
 - Framework must provide some 'useful' functionality related to the application domain
- **Extensibility**
 - Hot-spots designed to be extended
 - Allow developer to customize their application specifically for a particular purpose
- **Non-modifiable Framework Code**
 - Key components of the framework cannot be altered
 - Not strictly non-modifiable, but typically just used, though contributions back to the framework are often subject to the framework creators or open source community

Framework Characteristics

- **Advantages:**
 - Enables rapid development
 - Concentrate on unique application logic
 - Reduces boiler plate code
- **Disadvantages:**
 - Impose a certain model of development (80% easy / 20% hard)
 - Frameworks can introduce code bloat
 - Levels of abstraction generally introduce performance penalties
 - Difficult to overcome the steep learning curve
 - Traditionally poorly documented

Framework vs Libraries?

- A framework is about reusing behaviours by how abstract classes and components interact with each other
 - A framework calls your application code
- A library is a collection of classes which provide reusable functionality
 - Your application code calls the library

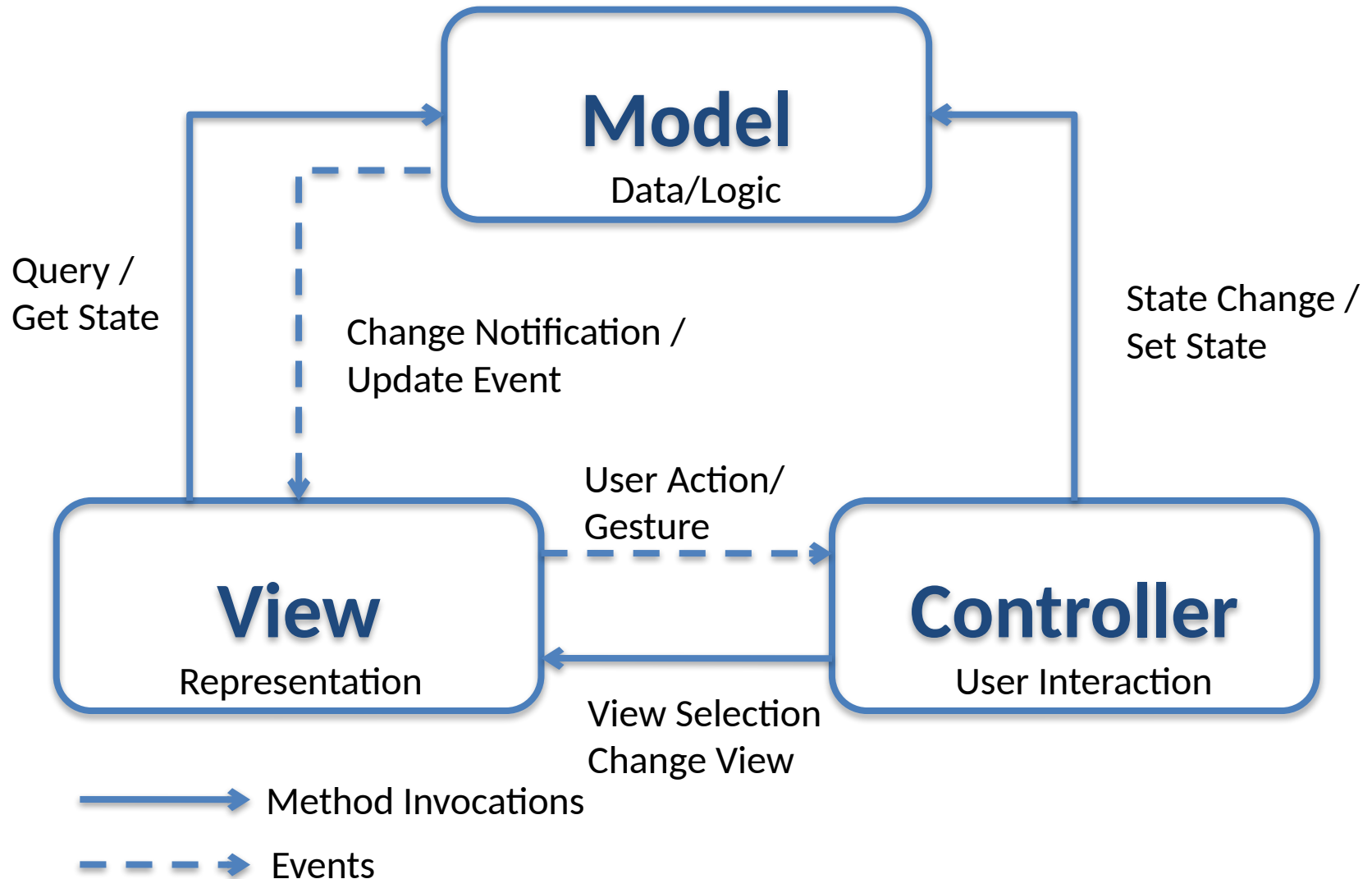
Degrees of Activeness

- **Passive Framework:**
 - Just a set of starting files that you unpack
 - And then use to build your application
- **Semi active frameworks**
 - Generates your starting project code
 - Then you customize and fill in the app logic
- **Active Frameworks**
 - They go beyond semi-active, and work in the background by generating additional code, without explicit commands from the developer

Pre-Fabricated Things

- Web frameworks typically provide (some of) the following features:
 - User authentication, authorization, security
 - Database abstraction (or Object-Relational Mapping)
 - Template system
 - Ajax sub-framework
 - Session Management
 - An Architecture usually based on **Model-View-Controller**

MVC Architecture



Model-View-Controller

- The model, this layer contains codes that operate on the application data.
 - Any actions that wanted to be executed on the raw data must go through this layer. Definitions of how the application work with data (commonly retrieve, create, update, or delete) are written here.
- The view, this is the presentation layer.
 - It defines how your pages should look like to the user, how the application presents data, or how a user can submit certain instructions to be executed by the application.
- The controller, this component acts as the orchestrator of the application.
 - It controls the flow of the program. It receives user commands, processes it, and then contacts the model, and finally instructs the view to display appropriately to the user.

MVC pros and cons

Advantages

- Enable independent development & testing
- Easier to maintain
- Provides reusable views & models
- Synchronized views and multiple simultaneous views
- Helps enforce logical separation of concerns

Disadvantages

- Some initial overheads splitting out concerns
- Increased overheads in development (i.e. 3 classes vs 1)
 - Especially for very simple applications
- Debugging it can sometimes be a problem
- Potential for excessive updates
- Requires the developers to think and understand patterns

Why use WAFs?

- To enable rapid development that matches the rapid release cycle of the web
- To reduce the development effort of programming in different languages/technologies
 - Database access (possibly Object-Relational Mapping)
 - Templating HTML
- To manage the complexity of the increasingly large and sophisticated web applications by including library support for:
 - User Authentication
 - Session Management
 - Creating a Web Service

Why use WAFs?

- To reduce 'boiler plate' code in web applications
 - Particularly access and manipulation of DB
 - Session management across multiple pages
 - Often referred to as CRUD operations
- Web-apps have matured to a point where software engineering practices (patterns (e.g. MVC) and frameworks) are becoming
 - increasingly useful,
 - necessary and
 - the norm.

Common Framework Functionality

- **Web Template System**
 - to provide pre-defined pages that load dynamic content
- **Caching**
 - to reduce perceived lag
- **Security**
 - to provide authentication and authorization functionality
- **Database access and mapping**
 - To speed up working with databases and avoid using SQL

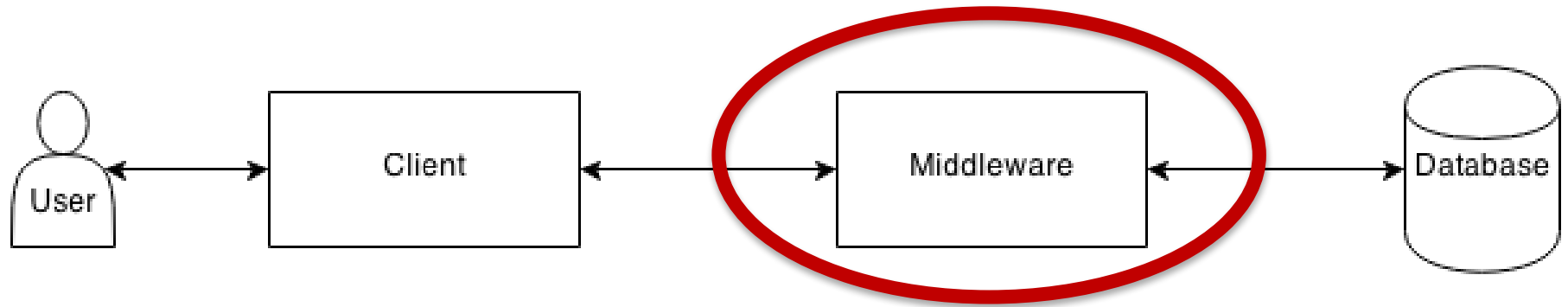
Common Framework Functionality

- **URL Mapping**
 - To enable handling of URLs and friendlier URLs
- **Ajax handlers and handling**
 - To create more dynamic pages that are more responsive
- **Automatic configuration**
 - To decrease the setup hassles, usually uses introspection and/or following conventions
- **Form Management**
 - To speed up the creation of forms and handling of forms

WAF Caveats

- They require an investment in learning the framework
 - **Learning vs. Building Trade-off**
- Sacrifice some flexibility for rapid development
 - **Flexibility vs. Efficiency Trade-off**
- Knowledge of one framework does not necessarily transfer to another
- Early stages of web framework eco-systems
 - There are many competing options at present
 - Eventually the most popular (few) will emerge

High Level System Architecture



- Web application frameworks sit in the middleware, for building the application server.

Many Web App Frameworks

- **Java**
 - Spring, Struts, Grails, Google Web Toolkit, Jboss, etc
- **PHP**
 - **Symfony**, Cake, Alloy, AppFlower, Lithium, Seagull, Solar, etc
- **Python**
 - **Django**, **Flask**, TurboGears, Pylons, Pyramid, Grok, Zope, etc
- **Ruby**
 - **Rails**, Camping, MonkeyBars, Merb, Ramaze, Nitro, etc
- **Perl**
 - Catlyst, Dancer, Reaction, Interchange, Mason, etc
- **ASP.NET**, ColdFusion, C++, Tcl, Javascript, Ocaml, Scala, Groovy, etc

http://en.wikipedia.org/wiki/Comparison_of_web_application_frameworks

django

Django

- Pronounced JANG-oh
- *“Django is a high-level Python Web framework that encourages rapid development and clean, pragmatic design”*
- Claimed to be the web framework for perfectionists with deadlines – you may differ!

Django

- A **Python** Based Web Application Framework
- Its primary goal is to ease the creation of complex, database-driven websites
- Emphasizes **reusability** and **pluggability** of components and **rapid development**
- Provides an optional administrative CRUD (create, read, update and delete) interface
 - This is created dynamically through introspection.
- Deviates from MVC employing **Model View Template**

Django

- History:
 - Created by Adrian Holovaty and Jacob Kaplan-Moss at World Online news for efficient development
 - Open Sourced in 2005, first major release in 2008
- Primary Focus:
 - Dynamic and database driven websites
 - Content based websites
 - Examples:
 - eBay, Amazon, GumTree, etc
 - The Guardian, Herald, etc
 - Match, Twitter, etc
- Django is being used by...
 - Instagram, Spotify, Disqus, YouTube, to name a few.

Why Django for Web Dev

- Lets you divide code modules into logical groups
 - Providing flexibility and easier to change
 - Underpinned by the MVC/MVT design pattern
- Provides automatically generated web administration
 - Easier to manage the database
- Provides many pre-packaged APIs for common tasks

Why Django for Web Dev

- Provides a template system to define HTML templates
 - Avoids code duplications
 - Subscribes to the Don't-Repeat-Yourself (DRY) principle
- Allows you to define what the URL will be for a given view
 - Loosely Coupled Principle
- Allows you to separate business logic from the presentation
 - Separation of Concerns

Overall Design Philosophy

- **Loose Coupling**
- **Less Code**
- **Quick Development**
- **Don't Repeat Yourself (DRY)**
- **Explicit is better than implicit**
 - A core Python principle
- **Consistency**
- See <http://docs.djangoproject.com/en/dev/misc/design-philosophies/> for more details and more philosophies

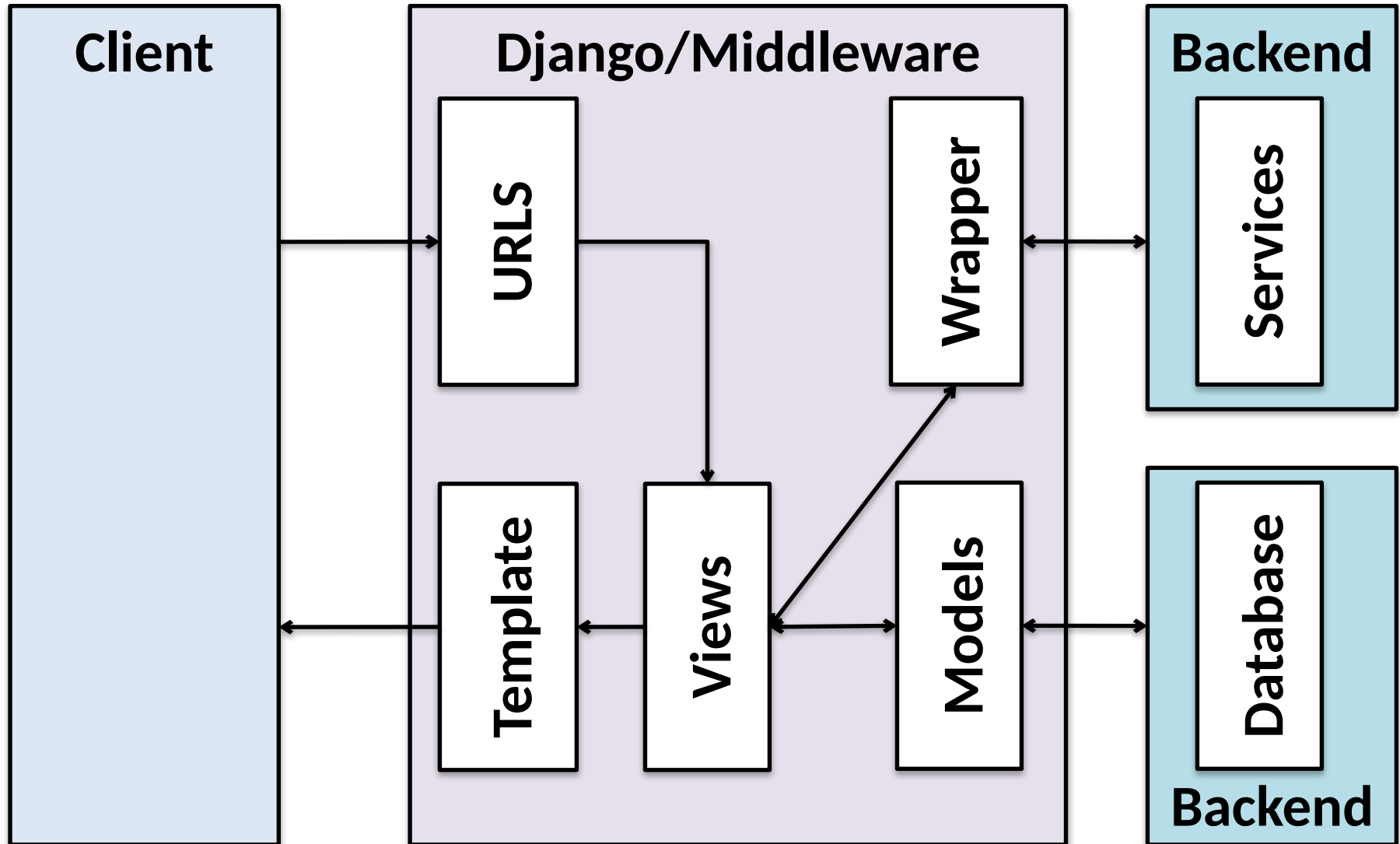
Django Modules

- Administration Interface (CRUD)
 - Create, Read, Update and Delete
- Authentication Systems
- Forms Handling
- Session Handling
- Syndication Frameworks
 - RSS and Atom feeds
- Caching
- Internationalization and Localization
- And much, much more...

Model View Controller in Django

- **Models** describe your database
- **Views** determines what the user sees
- **Controller** is handled by
 - the Django Framework
 - URL parser maps URLs to views
- **Templates** describe how the data is presented
 - Additional to the MVC, i.e. MVCT

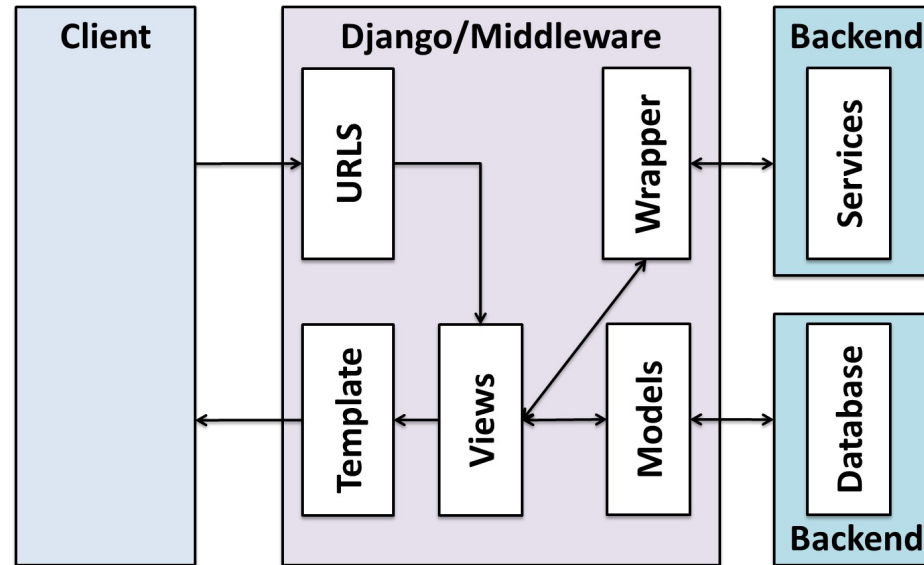
Simplified Internal Flow



Internal Sections/Components

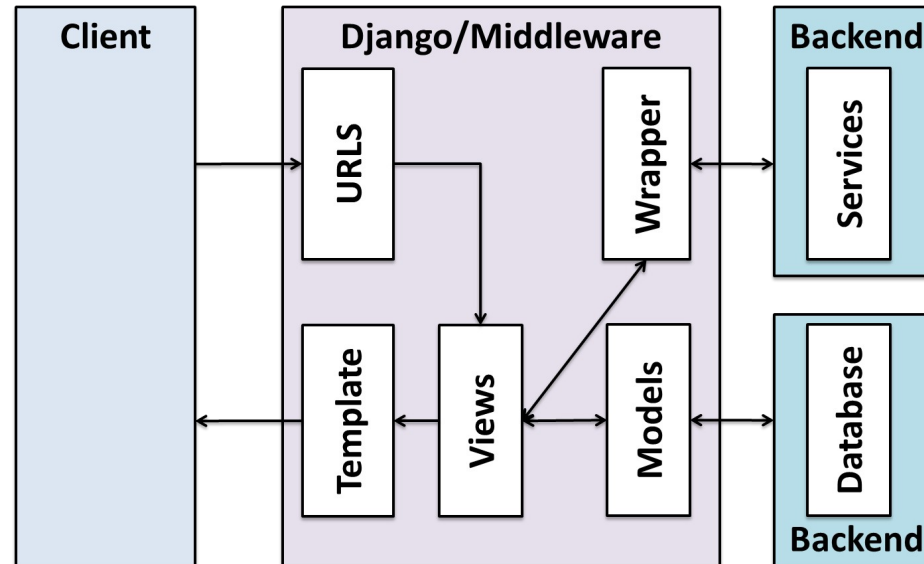
Branch: master ▾ rango2 / tango_with_django_project / rango /		
 gerac83 ch10 finished		
..		
 migrations		to be tested ch9
 __init__.py		ch3 page 38
 admin.py		to be tested ch9
 apps.py		ch3 page 38
 forms.py		to be tested ch9
 models.py		to be tested ch9
 tests.py		ch3 page 38
 urls.py		to be tested ch9
 views.py		ch10 finished

Internal Sections/Components



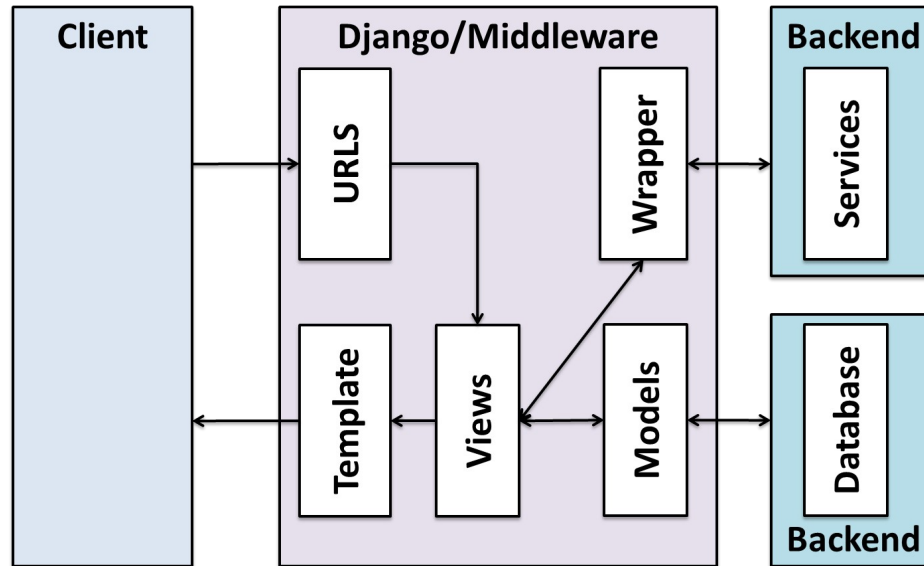
- **Building Data Models (models.py):**
 - The models specify the entities and relationships in the database – these provide an Object Relational Mapping to the actual database tables
 - Django constructs the database given the models defined

Internal Sections/Components



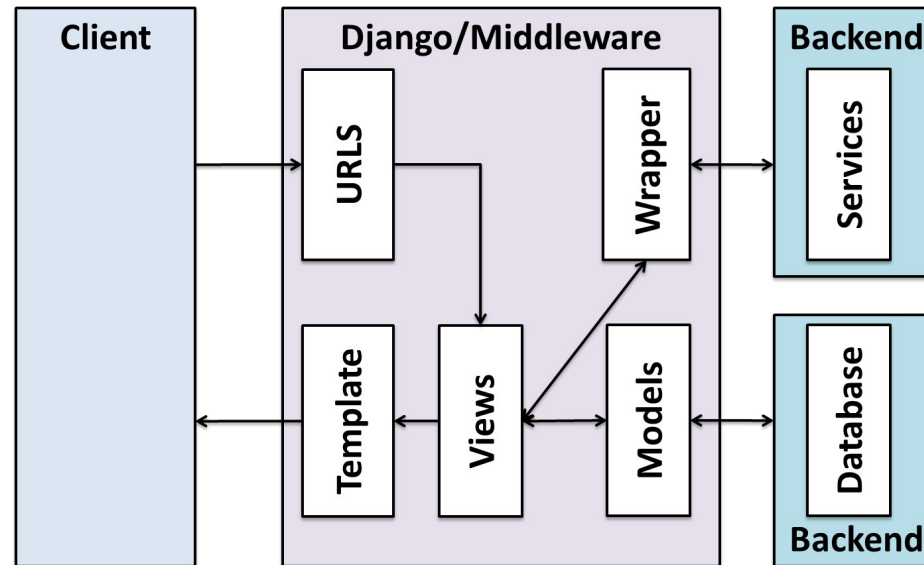
- **Defining Views (views.py):**
 - Views are responsible for handling and processing the specific request, collating the data from databases/external services, then selecting the template, for the response to be generated

Internal Sections/Components



- **Controlling flow (urls.py):**
 - To specify what view function should handle a particular URL (or part of), URL patterns are used to find matches with the URL, and to route this request to the appropriate view
 - The use of pattern matching means that different instantiations can be handled by a common pattern

Internal Sections/Components



- **Providing Templates**
 - The templates mean the response format (HTML, XML, etc) is decoupled from the data to be presented



Ruby on Rails

Sustainable productivity for web-application development

Ruby on Rails

- **A Ruby-based web framework**
 - Sparked interest in developing web applications rapidly
 - Development of competing frameworks in other languages
 - Made popular by “**Blog in 15 minutes**” and other screen casts
 - Web development could be:
 - A. Fast,
 - B. Fun,
 - C. Straightforward.

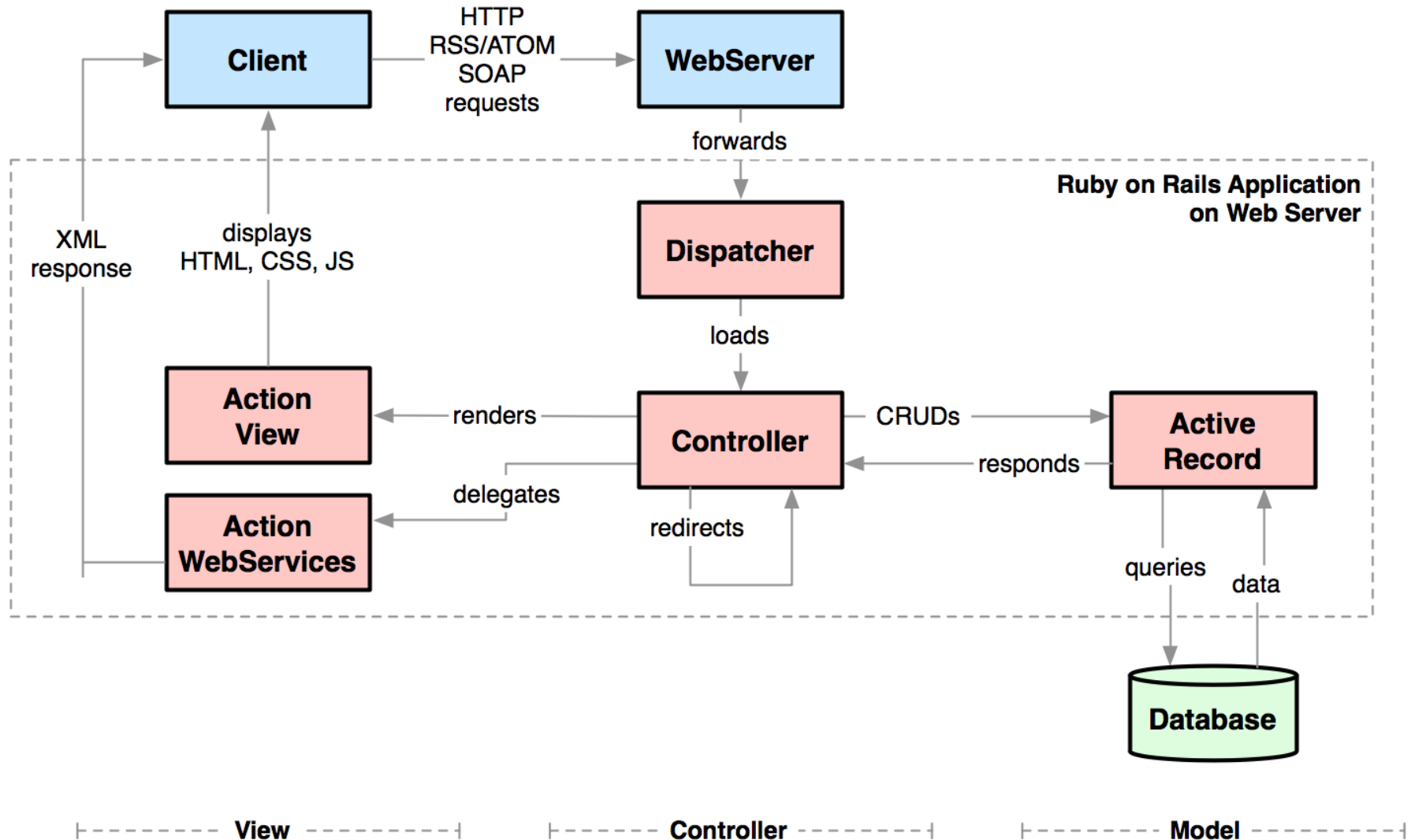
Rail Design

Employs several key design principles:

- **Convention over Configuration**
 - Decreases the number of decisions a developer needs to make
 - Only specify unconventional aspects of the application
- **Don't Repeat Yourself**
 - Store information only in one unambiguous place
- **Model-View-Controller**
 - A set of design patterns that allows the separation between data models, user interfaces and the control logics of the application

<https://speakerdeck.com/anildigital/solid-design-principles-in-ruby>

Ruby on Rails Architecture



Ruby on Rails Example

- Build a Blog in 8 minutes with Ruby On Rails
Ruby 2.4.2 Rails 5.1.4
 - <https://www.youtube.com/watch?v=U5j-InRWSNE>

symfony

Symfony

- A PHP based Web Application Framework
 - **Its main goal is to speed up the creation and maintenance of web applications and replace repetitive coding tasks**
 - Follows the MVC paradigm
 - Integrates with many existing technologies
 - Heavily inspired by WAFs such as:
 - Ruby on Rails,
 - Django, and
 - Spring.

From Full Frameworks to Lightweight Frameworks

- Django / Ruby on Rails: *"Everything included"*
 - ORM, templates, admin, auth, migrations
 - Great for large integrated apps
 - Heavy for small experiments or microservices
- Flask: *"Start small, add only what you need"*
 - A microframework – routing and a minimal core

So Many Frameworks...

A close-up photograph of a hand reaching into a clear glass jar. The jar is filled with various colorful, shaped cookies (biscuits). A hand is reaching in from the top, with fingers spread, about to pick up a brown, star-shaped cookie. Other visible cookies include a purple one, a yellow one with an orange beak, a blue one, and a pink one. The background is a plain, light-colored surface.

Which one to choose?

How to choose?

- Ruby On Rails and Django are all good in many aspects
- Documentation is excellent, with similar learning curves
- **Why go for Django?**
 - makes life easier in terms of the structuring of files/folders (less is more)
 - Has easier form management
 - Templates in Django are more flexible (i.e. have inheritance to extend them, built in tags, filters, etc)
 - Django uses Python which is great (Yahoo uses php, Google uses python)
 - Tends to be more minimalist (no frills)
 - Sometimes the documentation is too minimalist, which requires exploring the code.

But what about Ruby on Rails?

- **Ruby on Rails** is... more or less the same, but a different language.
 - It has a big community, where you can find almost everything
 - But, you often find what you need made by a clueless noob
 - Perhaps, offers a false sense of ability, that you can do anything without effort.
 - When you realize the truth..... it is often too late.
 - Which leads to many rules being broken to kludge things together

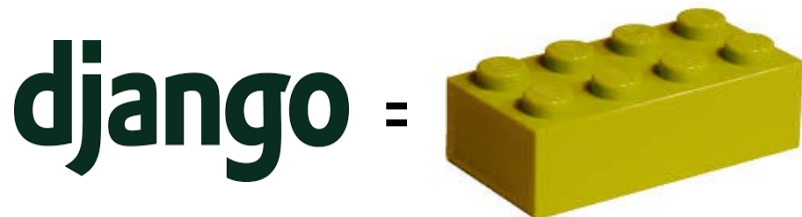


Flask

web development,
one drop at a time

Flask

- **Flask:** a lightweight web application framework in Python
- **Philosophy:** Minimal core, maximum flexibility
 - Lets you choose your own:
 - Database layer (SQLAlchemy, Peewee, etc.)
 - Auth system
 - Extensions (Flask-Login, etc.)



Why Flask

- **Lightweight:** start from the essentials, add components as needed/if needed
- **Easy to learn:** small framework, easy for beginners, quick to prototype
- **Customizable:** plug in an extension, get a new feature
- **Great for building APIs**

Django vs Flask

	Django	Flask
Popularity	High	High
Difficulty	Moderate	Low
Architecture	Clear structure	Easily extensible
Templates	Built-in template language	Jinja2 templates
Databases	Built-in ORM, supporting specific databases	ORM support through extensions
URLs	Powerful routing - complex	Routing and logic in one file - simple

Summary

- Using Web Application Frameworks saves you a lot of effort because they save you from coding common functionality repeatedly and integrate good SE practices
- There are a lot of WAFs out there to choose from, each with their own benefits and disadvantages
- But at the end of the day you can do more-or-less whatever you like with any of these frameworks, it all depends on your context.
- Django is a Python based framework that allows you to develop web applications quickly using modules and MVT